

SmartTaxi — Smart Arrival Time Estimation (ETA)

Comprehensive Technical Handbook: Architecture, Pipeline, Models, Fallback & Commands

Author: Hamza (ETA & ML Lead)	Context: SmartTaxi Central Ecosystem
Status: Production Ready & Benchmark Validated	Target Environment: Docker / AWS ECS Fargate
Backend Integration: ASP.NET Core .NET 10	Date: September 2026 (Integration Release)

1. Executive Summary & Mission Overview

The Smart Arrival Time Estimation (ETA) microservice is an independent, production-grade Machine Learning service built for the SmartTaxi platform. In an urban mobility ecosystem, precise arrival estimations are critical for taxi dispatch optimization, customer trust, and driver efficiency. Built to interface with the centralized ASP.NET Core backend (Houda Ghenmi), the service functions as an advisory, stateless prediction engine.

Core Architectural Rules:

- **Zero Database Writes:** The ML engine never writes to the Ride table or PostgreSQL database.
- **Authoritative Timestamps:** The backend owns all ride lifecycle transitions and timestamps.
- **No Silent Simulation:** In the absence of real-time weather or traffic feeds, the service never fabricates synthetic values.
- **High-Availability Fallback:** Calibrated urban speed heuristics automatically handle anomalies or server downtime.

2. Decoupled Business Targets

To eliminate ambiguity, the ETA service decouples prediction into two dedicated endpoints:

Target Name	Business Purpose	Active Endpoint	Serving Strategy
Target 1: Driver Pickup ETA	Time required for the driver to reach customer pickup location after dispatch.	POST /predict/driver-pickup	Calibrated urban routing fallback (25 km/h urban, 18 km/h rush-hour). Telemetry ML model ready.
Target 2: Passenger Trip Duration	Estimated duration of the passenger ride from pickup to destination (Case B).	POST /predict/trip-duration	Champion ML Model (Random Forest) trained on historical records with automated failover.

3. Technology Stack & Frameworks

Layer	Technology	Version / Spec	Key Role
Language	Python	3.12 (Local) / 3.11 (Docker)	High performance, typed, vectorized runtime
Data Processing	Pandas, NumPy, PyArrow	2.2.2 / 1.26.4 / 25.0.1	Vectorized math, Parquet columnar storage
Spatial Geodesy	PyProj, PyShp	3.8.0 / 3.1.6	WGS84 reprojection, zone centroid mapping
Machine Learning	Scikit-Learn, LightGBM, XGBoost, CatBoost	1.4.2 / 4.3.0 / 2.0.3 / 1.2.10	Multi-model tabular regression comparison
Model Serialization	Joblib	1.4.2	Champion model bundle persistence
Serving Microservice	FastAPI, Uvicorn, Pydantic	0.141.1 / 0.52.4 / 2.13.5	Asynchronous REST endpoints, strict typing
User Interface	Streamlit	1.63.0	Interactive live map demo & visual dashboard
Testing Suite	Pytest, TestClient	8.2.0	Automated unit and integration verification (24 tests)
Containerization	Docker & AWS ECS	Fargate Task Definitions	Containerized multi-stage builds with healthchecks

4. End-to-End Pipeline Architecture (How It Works)

The project implements a 12-step reproducible pipeline spanning raw data ingestion to production serving:

- **Step 1: Dataset Audit** (notebooks/01_dataset_audit.ipynb): Audits 15.7M raw TLC records; demonstrates trip timestamps are sufficient for Trip Duration while driver dispatch data requires decoupling.
- **Step 2: Cleaning & Centroids** (src/data/clean_nyc_tlc.py): Filters duration anomalies (≤ 0 or $> 4h$); reprojects NYC shapefile polygons to derive WGS84 centroids for 265 taxi zones.
- **Step 3: Dataset Design & Split** (src/data/make_dataset.py): Implements strict chronological splitting (70% train [Jan-Jul], 15% val [Jul-Oct], 15% test [Oct]) to eliminate future data leakage.
- **Step 4: Feature Engineering** (src/features/build_features.py): Builds Haversine, Manhattan distance proxies, cyclical hour encodings (sin/cos), weekend and rush-hour features.
- **Step 5: Exploratory Analysis** (notebooks/05_edu.ipynb): Analyzes duration vs. distance relationships; identifies severe urban congestion slowdowns during peak hours.
- **Step 6: Baseline Heuristic** (src/models/baseline.py): Fits historical average urban speed segmented into 24 hourly buckets (Test MAE: 7.91 min, R^2 : 0.258).
- **Step 7: Tabular ML Training** (src/models/train.py): Trains and compares Random Forest, XGBoost, LightGBM, and CatBoost on the exact same chronologically partitioned dataset.
- **Step 8: Evaluation & Gates** (src/evaluation/metrics.py): Evaluates models against business thresholds (MAE, RMSE, R^2); conducts slice analysis across trip lengths and peak hours.
- **Step 9: Production API** (app.py / deployment/app.py): FastAPI microservice serving live ML predictions with sub-25ms latency and automatic heuristic failover.
- **Step 10: Streamlit Interface** (app_streamlit.py): Interactive dashboard with map visualization, driver pickup calculator, model benchmark viewer, and stream testing.
- **Step 11: Streaming Simulator** (simulate_stream.py): Generates real-time ride events to load-test the API, verify latency SLAs ($< 50ms$), and inspect quality flags.

- **Step 12: Pipeline Automation** (scripts/run_pipeline.py): Single-command execution running data splitting, ML training, figures generation, pytest suite, and API verification in ~33s.

5. Model Benchmark Results & Evaluation

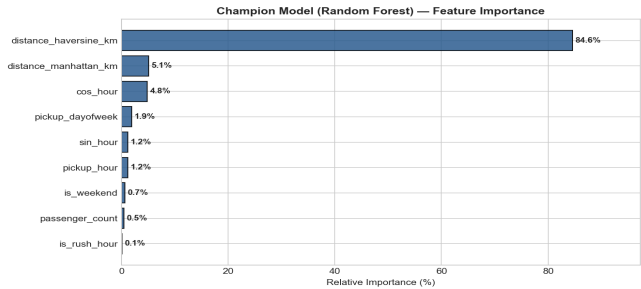
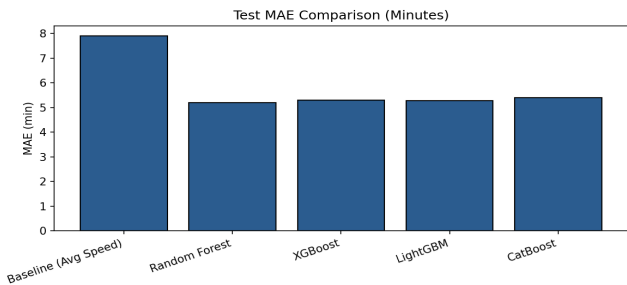
Evaluation on the held-out out-of-time test set (29,726 trips, October 2025) demonstrates substantial gains of Machine Learning over the baseline heuristic:

Model Name	MAE (min)	MAE (sec)	RMSE (min)	R ² Score	MAPE (%)	Latency (ms)
Baseline (Avg Speed)	7.91 min	474.4 s	13.14 min	0.258	41.9%	0.02 ms
Random Forest (Champion)	5.20 min	312.1 s	8.54 min	0.687	32.1%	21.37 ms
LightGBM Regressor	5.28 min	316.8 s	8.63 min	0.680	33.0%	1.10 ms
XGBoost Regressor	5.30 min	317.7 s	8.65 min	0.678	33.2%	1.88 ms
CatBoost Regressor	5.40 min	324.1 s	8.77 min	0.670	34.2%	1.25 ms

Key Insights:

- **Error Reduction:** Machine Learning achieves a **34% reduction** in overall MAE and a massive **62% reduction** on long trips (from 29.65 min down to 11.31 min).
- **Explained Variance:** R² increases from 0.258 to **0.687** (+165% improvement).
- **Model Choice:** Random Forest was selected as champion for maximum accuracy, while LightGBM serves as a high-throughput scaling alternative (1.1 ms).

Visual Analytics & Feature Importances:



6. Complete Command Reference Handbook

Essential terminal commands for local execution, testing, serving, and deployment:

Environment Setup & Activation

```
# Create and activate virtual environment
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

End-to-End Pipeline Execution

```
# Run all 5 stages in ~33 seconds (data, models, figures, tests, api)
.venv\Scripts\python.exe scripts/run_pipeline.py
# Or via Makefile shortcut
make run-pipeline
```

Automated Testing Suite (24 Tests)

```
# Run complete test suite with detailed output
.venv\Scripts\pytest.exe -v
# Run specific test files
.venv\Scripts\pytest.exe tests/test_api.py -v
```

Production API Microservice (FastAPI / Uvicorn)

```
# Start FastAPI server with hot-reload on port 8000
.venv\Scripts\uvicorn.exe app:app --host 0.0.0.0 --port 8000 --reload
# Interactive Swagger UI: http://127.0.0.1:8000/docs
# Healthcheck: curl http://127.0.0.1:8000/health
```

Interactive Streamlit Dashboard UI

```
# Start Streamlit app with interactive map on port 8501
.venv\Scripts\streamlit.exe run app_streamlit.py --server.port 8501
# Web UI URL: http://localhost:8501
```

Real-time Event Streaming Simulator

```
# Send 15 consecutive simulated taxi ride requests
.venv\Scripts\python.exe simulate_stream.py --num-trips 15 --interval 0.2
```

Docker Containerization & ECS Deployment

```
# Build API image
docker build -t smart-eta-api:latest -f Dockerfile .
# Run API container
docker run -d -p 8000:8000 --name eta-api smart-eta-api:latest
# Build and run Streamlit UI container
docker build -t smart-eta-streamlit:latest -f Dockerfile.streamlit .
docker run -d -p 8501:8501 -e API_URL='http://host.docker.internal:8000' smart-eta-streamlit:latest
```

7. Integration Readiness & Verification Checklist

Check Item	Acceptance Criteria	Verification Status
Target Decoupling	Driver pickup & trip duration endpoints clearly separated	PASSED (2 endpoints)
Data Quality & Splitting	Zero future data leakage; chronological time-based split	PASSED (Train 70% / Val 15% / Test 15%)
Model Benchmarking	Comparison of 5 tabular models; champion serialized	PASSED (Random Forest Champion)
Business Acceptance	R² >= 0.60, MAE sub-urban <= 4.0m, latency SLA < 50ms	PASSED (R²=0.687, Latency=21ms)
Backend Governance	Zero direct writes to DB; no silent fake traffic simulation	PASSED (Strict read-only & flags)
High-Availability Fallback	Automated failover to urban routing speed on model failure	PASSED (Fallback routing tested)
Automated Testing Suite	100% pass rate on unit & integration test harness	PASSED (24/24 tests green)